

Practical No:-03

Implement a singly linked list with basic operations: insertion, deletion, and traversal.

```
#include <iostream>
using namespace std;

// Create a node structure
struct Node {
    int data;
    Node* next;
};

// Head pointer
Node* head = NULL;

// Function to insert at the end
void insert(int value) {
    Node* newNode = new Node();
    newNode->data = value;
    newNode->next = NULL;

    // If list is empty
    if (head == NULL) {
        head = newNode;
    } else {
        Node* temp = head;
        while (temp->next != NULL) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
    cout << "Inserted: " << value << endl;
}

// Function to delete a node
void deleteNode(int value) {
```

```

Node* temp = head;
Node* prev = NULL;

// If first node is to be deleted
if (temp != NULL && temp->data == value) {
    head = temp->next;
    delete temp;
    cout << "Deleted: " << value << endl;
    return;
}

// Search for the value
while (temp != NULL && temp->data != value) {
    prev = temp;
    temp = temp->next;
}

// If value not found
if (temp == NULL) {
    cout << "Value not found!" << endl;
    return;
}

prev->next = temp->next;
delete temp;
cout << "Deleted: " << value << endl;
}

// Function to display the list
void display() {
    Node* temp = head;
    cout << "Linked List: ";
    while (temp != NULL) {
        cout << temp->data << " -> ";
        temp = temp->next;
    }
    cout << "NULL" << endl;
}

```

```
// Main function
```

```
int main() {
```

```
    insert(10);
```

```
    insert(20);
```

```
    insert(30);
```

```
    display();
```

```
    deleteNode(20);
```

```
    display();
```

```
    return 0;
```

```
}
```

Output:

Inserted: 10

Inserted: 20

Inserted: 30

Linked List: 10 -> 20 -> 30 -> NULL

Deleted: 20

Linked List: 10 -> 30 -> NULL

